

Bootcamp: Full Stack Junior

Modulo 2: Ingeniería de Software y Análisis del Negocio

Semana 4

Contenido

1

Comprensión del negocio y stakeholders

2

Análisis y levantamiento de requerimientos

3

Historias de usuario

4

Criterios de aceptación

5

Priorización y gestión del backlog

6

Introducción a contratos técnicos de API

7

Diagramación UML (clases)

8

Diagramas de arquitectura de software

Contenido:

1. ¿Por qué la Ingeniería de Software empieza en el negocio?

1.1. Relación entre problema de negocio y solución tecnológica

En el ámbito tecnológico, esta relación se define como un puente de valor. No existe una solución técnica exitosa si no nace de una necesidad real de negocio.

- **El Problema (El "Por qué"):** Es una brecha, dolor o ineficiencia que impide a una organización alcanzar sus objetivos (ej. "Perdemos el 20% de clientes en el carrito de compras").
- **La Solución (El "Con qué"):** Es la implementación técnica (software, automatización, IA) diseñada específicamente para cerrar esa brecha.
- **El Riesgo:** Implementar tecnología "porque está de moda" sin entender el problema base genera desperdicio de recursos (famoso efecto de "un martillo buscando un clavo").

1.2. Rol del Analista en Equipos Ágiles

En entornos ágiles (Scrum, Kanban), el analista actúa como el **puente de comunicación** y el guardián del valor. Sus responsabilidades principales son:

- **Refinamiento del Backlog:** Transformar necesidades ambiguas en Historias de Usuario claras, pequeñas y testeables.
- **Facilitador de Criterios de Aceptación:** Definir qué significa "terminado" para asegurar que el equipo técnico construya lo que el negocio espera.
- **Traductor Contextual:** Asegurar que los desarrolladores entiendan por qué detrás de cada tarea y que los Stakeholders entiendan las limitaciones técnicas.
- **Mitigador de Ambigüedad:** Resolver dudas en tiempo real durante el sprint para evitar bloqueos.



Ilustración 1 - Diagrama de Scrum team - Fuente: Shutterstock

1.3. Importancia de entender "QUÉ" se necesita antes de construir "CÓMO"

Este es el principio fundamental para evitar el retrabajo y el fracaso de proyectos.

- **El QUÉ (Requerimiento/Necesidad):** Define el objetivo. *Ejemplo: "Necesito que mis clientes puedan pagar sus facturas desde el móvil".*
- **El CÓMO (Diseño/Implementación):** Define la técnica. *Ejemplo: "Usar una API de Stripe integrada en una app nativa de iOS".*

¿Por qué este orden es vital?

1. **Flexibilidad:** Si entiendes el "Qué", puedes encontrar múltiples "Cómos" (quizás un sitio web móvil es más barato que una app nativa).
2. **Reducción de Costos:** Cambiar una frase en un documento de requisitos es barato; cambiar una arquitectura de software ya programada es carísimo.
3. **Alineación:** Evita que el equipo construya una solución técnicamente perfecta para un problema que no existe.

Todo el ecosistema del desarrollo de un producto debe de cumplir determinados pasos antes de comenzar con la construcción de dicho producto por eso es importante entender lo ciclos de desarrollo de soluciones.



Ilustración 2 - Ciclos de desarrollo de soluciones - Fuente: Autor de contenidos

2. ¿Qué es un stakeholder?

Un stakeholder es cualquier individuo, grupo u organización que pueda afectar, verse afectado o percibirse a sí mismo como afectado por una decisión, actividad o resultado de un proyecto.

No solo son "los jefes"; es cualquier persona que tenga algo que ganar o perder con la solución tecnológica que se va a construir.

Clasificación Fundamental

Para entenderlos mejor, solemos dividirlos en dos grandes grupos:

1. Stakeholders Internos

Están directamente vinculados a la organización que ejecuta el proyecto.

- **Patrocinador (Sponsor):** Quién da el presupuesto y la aprobación final.
- **Equipo de Desarrollo:** Programadores, diseñadores y testers que construyen el "cómo".

- **Gerentes de Producto/Líderes de Área:** Quienes definen las reglas del negocio.

2. Stakeholders Externos

No pertenecen a la empresa, pero su influencia es crítica.

- **Clientes/Usuarios Finales:** Quienes usarán la tecnología diariamente.
- **Proveedores:** Entidades externas que brindan servicios necesarios (ej. servicios de nube).
- **Reguladores:** Entidades gubernamentales o legales que imponen normas (ej. leyes de protección de datos).

3. Escritura de Historias de Usuario y Criterios de Aceptación

Una Historia de Usuario es una explicación informal y general de una funcionalidad de software escrita desde la perspectiva del usuario final. Su propósito es articular cómo una función de software proporcionará valor al cliente.

La forma más común de escribirlas es:

Como [tipo de usuario], quiero [acción o funcionalidad], para [beneficio o resultado esperado].

Por ejemplo:

Como huésped, quiero poder filtrar alojamientos por precio y ubicación, para encontrar opciones acordes a mi presupuesto y preferencias.

Esta forma sostiene el enfoque en el usuario y obliga a desarrollar en la razón detrás de cada funcionalidad.

Se basan en las "3 C's":

1. **Card (Tarjeta):** La descripción escrita (el "qué").
2. **Conversation (Conversación):** Las discusiones entre el equipo y el Product Owner para aclarar detalles.
3. **Confirmation (Confirmación):** Los criterios de aceptación que verifican que está terminada.

Componentes de la historia de usuario



Tarjeta

La descripción escrita de la historia de usuario.



Conversación

Las discusiones entre el equipo y el Product Owner.



Confirmación

Los criterios de aceptación que verifican que está terminada.

Ilustración 3 - 3c's - Fuente: Autor de contenidos

Criterios de Aceptación

Los Criterios de Aceptación son declaraciones claras y verificables que definen las condiciones que deben cumplirse para que la Historia de Usuario sea considerada terminada. Actúan como una especie de “contrato” entre el Product Owner y el equipo.

Características clave:

- Específicos: describen comportamientos concretos del sistema.
- Medibles: permiten comprobar objetivamente si se cumplen.
- Independientes: cada criterio evalúa un aspecto particular.
- Enfocados en el usuario: reflejan lo que el usuario experimentará.

Ejemplo (para la historia anterior):

Historia: Como huésped, quiero filtrar alojamientos por precio y ubicación, para encontrar opciones acordes a mi presupuesto y preferencias.

En conclusión, todos los pasos planteados anteriormente son de vital importancia para

En conclusión, resulta fundamental seguir cada uno de los pasos descritos para garantizar la correcta redacción y validación de las Historias de Usuario dentro del proyecto. Con este propósito, se adjunta un repositorio público [aquí](#) con una plantilla de documento como recurso adicional, destinada a facilitar la documentación, mantener la consistencia en la comunicación del equipo y asegurar que cada historia aporte valor real al usuario final.

4. Gestión y priorización del Backlog

El Product Backlog es el corazón del desarrollo ágil, conectando directamente las Historias de Usuario (con sus Criterios de Aceptación) con la planificación real del equipo. Representa la "lista maestra" de todo lo que se necesita hacer, priorizada para entregar valor máximo al usuario final.

4.1. Que es el Product Backlog

El Product Backlog es una lista dinámica y priorizada de todos los elementos pendientes para el desarrollo del producto, como Historias de Usuario, bugs, mejoras técnicas o investigaciones.

A diferencia de un backlog estático, evoluciona constantemente según feedback de usuarios, cambios de negocio o nueva información, siguiendo el principio ágil de adaptabilidad.

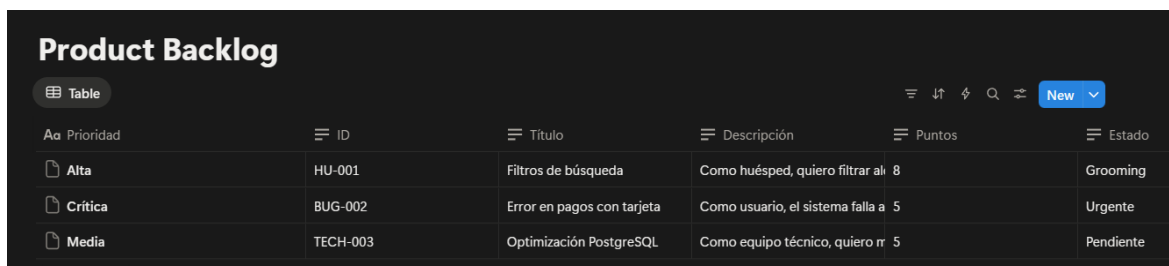
Sirve como fuente única de trabajo para el equipo, desde donde se seleccionan items para cada Sprint (Sprint Backlog).

Contenido Típico

Contiene Product Backlog Items (PBIs) variados, todos trazables a valor del usuario:

- Historias de Usuario (con sus Criterios de Aceptación ya validados).
- Correcciones de bugs críticos reportados por usuarios reales.
- Tareas técnicas (refactor, deuda técnica, infraestructura).
- Investigaciones/Spikes (prototipos para validar supuestos).
- Mejoras de rendimiento o accesibilidad.

Ejemplo en contexto de plataforma de reservas:



Prioridad	ID	Título	Descripción	Puntos	Estado
Alta	HU-001	Filtros de búsqueda	Como huésped, quiero filtrar al	8	Grooming
Crítica	BUG-002	Error en pagos con tarjeta	Como usuario, el sistema falla a	5	Urgente
Media	TECH-003	Optimización PostgreSQL	Como equipo técnico, quiero r	5	Pendiente

Ilustración 4 - Ejemplo del backlog - Fuente: Autor de contenidos

Cada item incluye: título, descripción, prioridad, estimación (Story Points) y criterios de aceptación cuando aplica

Quién lo Gestiona

Product Owner (PO) es el responsable único de:

- Priorizar items según valor de negocio/usuario.
- Añadir/eliminar elementos basándose en feedback.
- Aclarar detalles durante la Conversación de las 3 C's.

- Mantenerlo refinado (Backlog Grooming/Refinement).

El equipo de desarrollo colabora en:

- Estimaciones de esfuerzo (Planning Poker).
- Refinamiento (detallar Criterios de Aceptación).
- Retroalimentación sobre viabilidad técnica.

Conexión con Historias de Usuario:

Las Historias de Usuario bien escritas (usando el recurso de la plantilla en GitHub .md) se convierten en PBIs priorizados. Los Criterios de Aceptación definen el Acceptance Criteria del ítem en el backlog, creando trazabilidad completa desde la necesidad del usuario hasta la implementación.

5 Arquitecturas de Software Robustas y Escalables

En el desarrollo de software moderno, construir una aplicación que "funcione" es solo el primer paso. El verdadero reto para un Ingeniero de Software y Analista de Negocio radica en crear sistemas robustos (que no fallen ante errores) y escalables (que crezcan sin degradar su rendimiento).

Una arquitectura bien planificada es el puente entre los requerimientos del negocio y la viabilidad técnica a largo plazo.

5.1 Contratos de API: El Acuerdo de Interoperabilidad

En términos técnicos, un contrato de API es una **especificación técnica compartida** (usualmente en formato JSON a través de herramientas como OpenAPI/Swagger) que define las reglas de interacción entre un proveedor (backend) y un consumidor (frontend, app móvil u otro microservicio).

Imagina un contrato legal: si una parte cumple con enviar lo pactado, la otra está obligada a responder según lo acordado. El contrato oculta la complejidad; al consumidor no le importa si el backend está escrito en PHP, Node.js o Go, siempre que el contrato se respete.

Importancia de los Contratos de API

El uso de contratos transforma el ciclo de vida del desarrollo (SDLC) de las siguientes maneras:

- **Desacoplamiento (Parallel Development):** Una vez definido el contrato, el equipo de Frontend puede simular la API (usando *Mock Servers*) y empezar a programar sin esperar a que el Backend termine la implementación real.
- **Reducción de la Deuda Técnica:** Al tener reglas claras, se evitan parches de última hora debidos a que un campo cambió de nombre o de tipo de dato sin aviso.

- **Documentación Viva:** El contrato es la fuente de verdad. Herramientas como Swagger generan interfaces interactivas donde los desarrolladores pueden probar la API en tiempo real.
- **Estabilidad en Integraciones:** Facilita las pruebas de integración automatizadas (Consumer-Driven Contract Testing), donde se verifica que los cambios en el código no rompan el acuerdo establecido.

Elementos Clave de un Contrato de API

Para que un contrato sea funcional, debe desglosarse en los siguientes componentes técnicos:

Elemento	Descripción Técnica	Ejemplo
Endpoint	La URL única que identifica un recurso o acción.	/api/v1/users/42
Método HTTP	Define la intención de la operación (Semántica REST).	GET (Leer), POST (Crear), PUT (Actualizar)
Request (Petición)	Estructura de los datos enviados: Headers (auth), Query Params, y Body.	{ "name": "Juan", "email": "j@test.com" }
Response (Respuesta)	Estructura del objeto devuelto y tipos de datos.	{ "id": 42, "status": "active" }
Códigos HTTP	Indican el éxito o error de la petición mediante estándares.	200 OK, 201 Created, 400 Bad Request, 404 Not Found
Versionado	Estrategia para evolucionar la API sin romper aplicaciones antiguas.	.../v1/... vs .../v2/...

Tabla 1 - Elementos de un contrato API - Fuente: Autor de contenidos

6 Diagramas UML: Estructurando la Realidad

UML (Unified Modeling Language) es un lenguaje estándar para modelar visualmente la estructura y el comportamiento de un sistema, facilitando la comunicación entre perfiles técnicos y no técnicos.

Los Diagramas UML representan el lenguaje visual universal de la ingeniería de software y se clasifican en diferentes tipos. Pero entre todo el ecosistema de diagramas de UML ofrece los más comunes son los siguientes:

Diagramas de Casos de Uso (El "Qué")

Antes de programar, debemos entender qué quiere el usuario. Este diagrama define el alcance del sistema. Los elementos de diagrama de caso de uso son:

- **Actores:** Entidades externas que interactúan con el sistema (Usuarios, Otros Sistemas, Tiempo).
- **Casos de Uso:** Las funcionalidades o acciones (Ej: "Realizar Pago", "Iniciar Sesión").
- **Relaciones:** * <<include>>: Una acción que es obligatoria para otra (Ej: Para "Pagar" se *incluye* "Validar Fondos").
 - <<extend>>: Una acción opcional o condicional (Ej: "Aplicar Cupón" *extiende* a "Pagar").

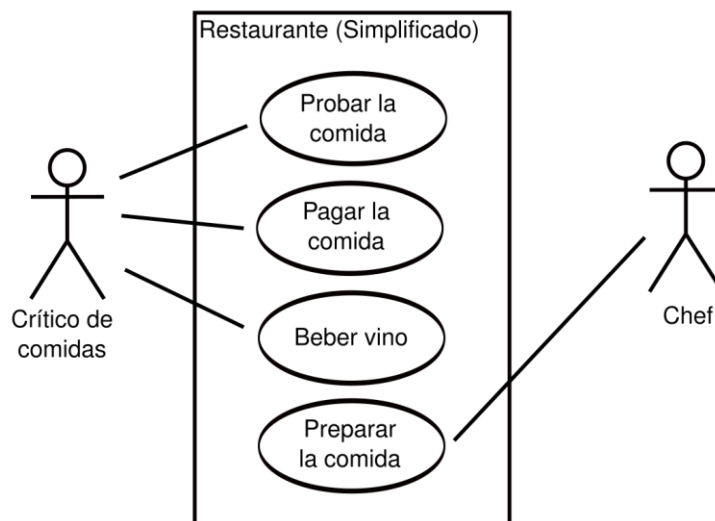


Ilustración 1- Ejemplo diagrama de caso de uso - Fuente: Wikipedia

Diagramas de Clases (El "Cómo está hecho")

Es el plano arquitectónico del software. Mientras que otros diagramas muestran acciones, esta muestra la estructura fija: qué componentes existen, qué datos guardan y cómo se relacionan entre sí. Los elementos que componen un diagrama de clases son:

Anatomía de una Clase

Se representa con un rectángulo dividido en tres niveles críticos:

1. **Nombre:** Define la entidad (siempre en singular y CamelCase, ej: UsuarioSoporte).
2. **Atributos (Estado):** Son las variables o datos que la clase conoce sobre sí misma.

3. **Métodos (Comportamiento):** Son las funciones o acciones que la clase puede ejecutar.

Modificadores de Visibilidad

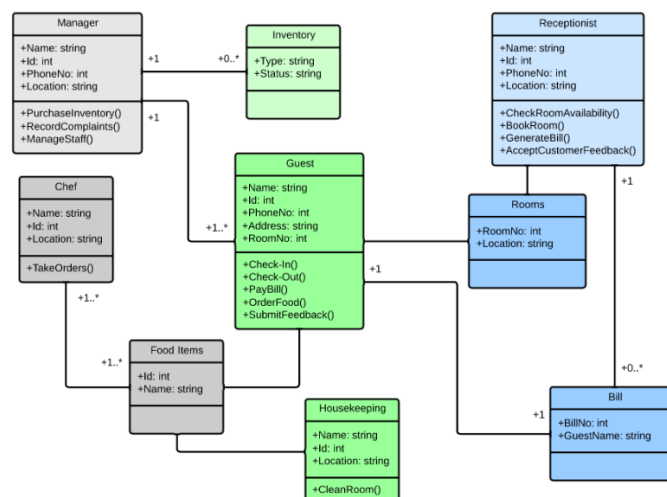
Indican quién tiene permiso para acceder a la información, un concepto clave para el **encapsulamiento**:

- **+ Público:** Acceso desde cualquier parte del sistema.
- **- Privado:** Solo la propia clase puede verlo (estándar para atributos).
- **# Protegido:** Solo la clase y sus "hijos" (herencia) tienen acceso.

Tipos de Conexiones (Relaciones)

Las líneas entre clases definen la lógica del negocio:

- **Herencia (Is-a):** Una clase hija hereda de una padre (ej: Perro es un Animal). Se usa una flecha con punta de triángulo vacío.
- **Asociación (Uses-a):** Una relación básica donde una clase usa a otra (ej: Cliente tiene una Dirección).
- **Composición (Part-of):** Una relación fuerte donde la parte no existe sin el todo (ej: un Libro y sus Páginas). Se usa un diamante negro.
- **Agregación (Has-a):** Una relación débil donde la parte puede existir sola (ej: un Equipo y sus Jugadores). Se usa un diamante vacío.



Diagramas de Secuencia (El "Cómo fluye")

Es el diagrama dinámico más importante para un desarrollador. Muestra la interacción entre objetos en un orden cronológico.

- **Líneas de Vida (Lifelines):** Representan las instancias de los objetos.
- **Mensajes:** Flechas que indican llamadas a métodos o respuestas de datos.
- **Activación:** Rectángulos sobre la línea de vida que indican que el objeto está ejecutando un proceso.

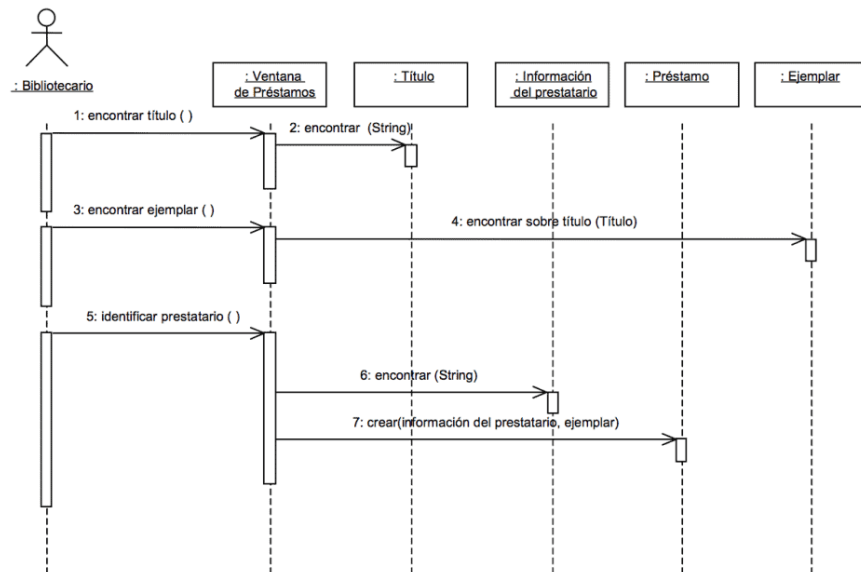


Ilustración 3- Ejemplo diagrama de secuencia- Fuente: manuel.cillero.es

7 Arquitectura en Capas y Desacoplamiento

La Arquitectura en Capas consiste en dividir el software en "pisos" lógicos. La regla de oro es: una capa solo puede conocer a la capa que tiene inmediatamente debajo, nunca a la que está arriba. Esto crea un sistema ordenado y predecible.

Capas de la Arquitectura Hexagonal

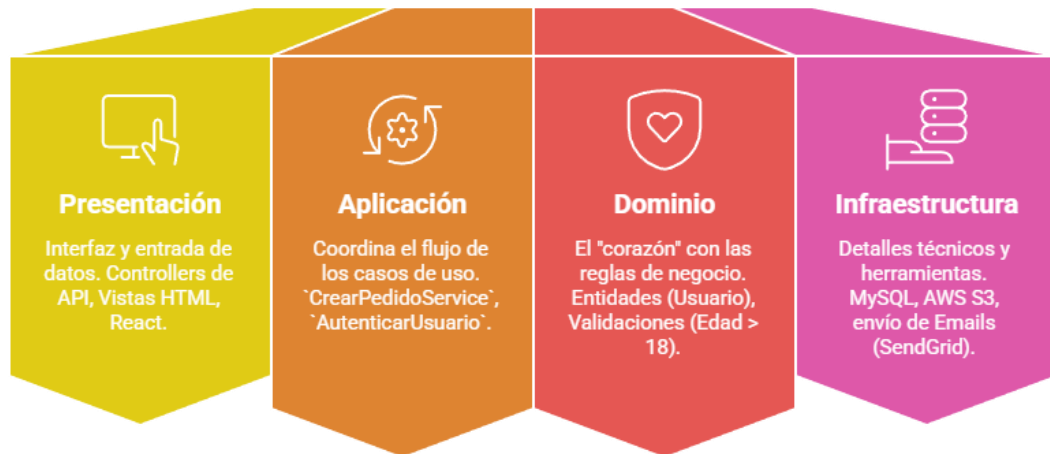


Ilustración 4- Responsabilidades por Capa - Fuente: Autor de contenidos

El Concepto de Desacoplamiento

El desacoplamiento es la capacidad de cambiar una pieza del sistema sin romper las demás.

- **Ejemplo:** Si tu arquitectura está bien desacoplada, podrías cambiar tu base de datos de **MySQL** a **MongoDB** modificando solo la capa de *Infraestructura*, sin tocar una sola línea de la lógica de negocio (*Dominio*).

Balance de la Arquitectura

Beneficios (Por qué usarla):

- **Mantenimiento:** Si hay un error en la base de datos, sabes exactamente que el problema está en *Infraestructura*.
- **Testabilidad:** Puedes probar las reglas de negocio sin necesidad de conectarte a una base de datos real (usando *Mocks*).
- **Paralelismo:** Un desarrollador puede trabajar en el Frontend mientras otro configura la persistencia de datos.

Riesgos (Qué evitar):

- **Dependencias Circulares:** Cuando la capa de Dominio intenta llamar a la de Presentación (rompe la jerarquía).
- **Lógica "Anémica":** Dejar el *Dominio* vacío y poner todas las reglas de negocio en los controladores de la API.
- **Sobrediagnóstico:** Crear demasiadas capas para un proyecto muy pequeño (aumenta la complejidad innecesariamente).

8 Arquitectura Hexagonal (Ports & Adapters)

La idea central es simple: El código que contiene la lógica de tu negocio debe estar en el centro y no debe depender de nada externo (ni de la base de datos, ni del framework, ni de la interfaz).

La conexión con los temas anteriores

- **De Capas a Hexágono:** En lugar de capas arriba/abajo, aquí hablamos de **Interior** (Lógica) y **Exterior** (Infraestructura).
- **Uso de Contratos de API:** El hexágono usa "Puertos" (interfaces) que funcionan como contratos. Si algo externo quiere hablar con el centro, debe cumplir con ese contrato.

Los 3 Componentes Clave

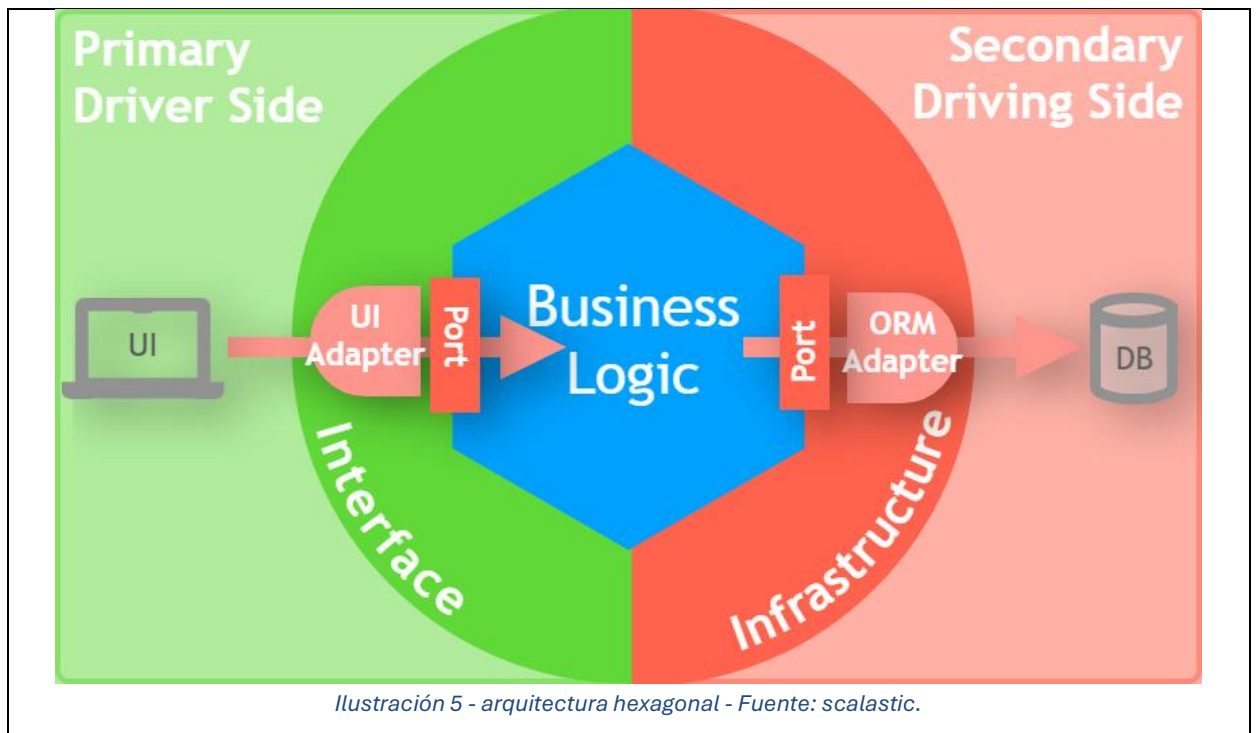
1. **El Núcleo (Core/Dominio):** Es el centro del hexágono. Aquí están las reglas de negocio (lo que aprendimos en el Diagrama de Clases). No sabe si los datos vienen de una web o de un archivo Excel.
2. **Puertos (Ports):** Son los "enchufes". Son interfaces que definen *qué* necesita el núcleo para funcionar (ej: "necesito guardar un usuario").
3. **Adaptadores (Adapters):** Son los "cables" que conectan el mundo exterior a los enchufes.
 - a. Un adaptador para **MySQL**.
 - b. Un adaptador para **MongoDB**.
 - c. Un adaptador para una **API REST**.

¿Por qué es mejor para el negocio? (Desacoplamiento total)

Imagina que tu empresa decide cambiar de proveedor de base de datos de Oracle a PostgreSQL:

- **Sin esta arquitectura:** Tendrías que reescribir casi todo el sistema.
- **Con Arquitectura Hexagonal:** Solo creas un nuevo **Adaptador**. El núcleo (tu lógica de negocio) no cambia ni una sola línea de código.

En resumen: La arquitectura hexagonal protege el corazón de tu aplicación de los cambios tecnológicos externos.



Recursos de Apoyo



Material de Lectura

[Plantilla documento historias de usuarios y requerimientos.](#)



Tutorial

[Tutorial de que es y cómo crear un product backlog.](#)



Ejercicio

[Ejercicio práctico para crear un product backlog.](#)